

**LAPORAN PRAKTIKUM SISTEM OPERASI
MANAJEMEN PROSES DAN CHAT IPC**



Dosen Pengampu:

Triawan Adi Cahyanto M.Kom

Disusun Oleh:

Yusa Salasa (2410651018)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

I. Tujuan Praktikum

1. Memahami konsep dasar proses dan mendapatkan Process ID (PID) menggunakan `getpid()`.
2. Memahami mekanisme pembuatan proses anak (**child process**) menggunakan fungsi `fork()`.
3. Memahami pentingnya dan cara kerja sinkronisasi proses menggunakan fungsi `wait()` untuk menunggu proses anak selesai.
4. Memahami mekanisme Komunikasi Antar Proses (IPC) dasar menggunakan **pipe** dan lanjutan menggunakan **FIFO (Named Pipe)** serta **Shared Memory**.
5. Menganalisis kode untuk membandingkan kinerja relatif antara mekanisme IPC yang berbeda.

II. Dasar Teori Singkat

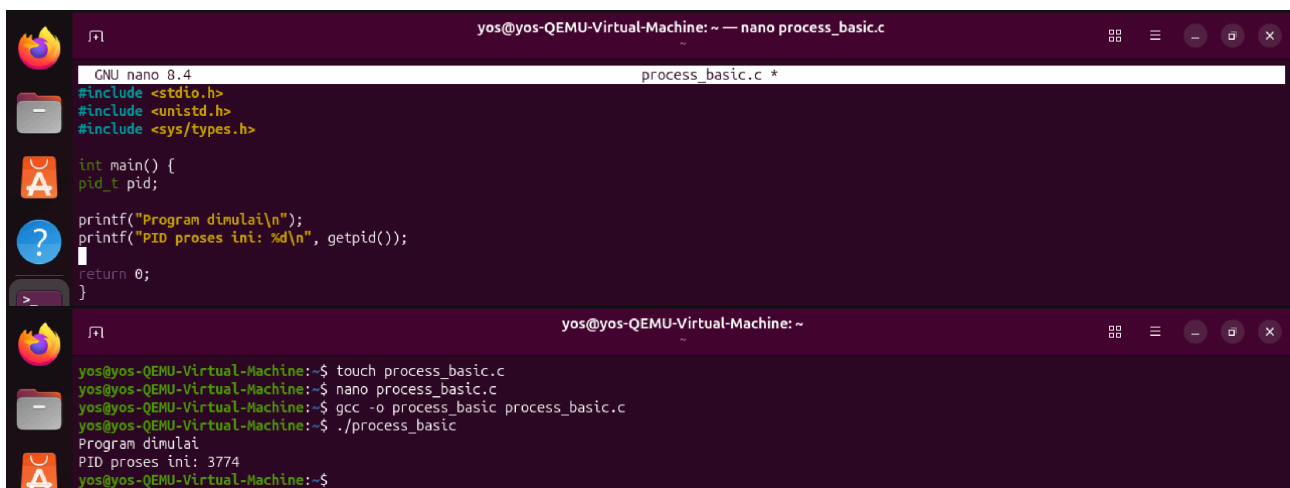
Proses adalah program yang sedang dieksekusi, diidentifikasi oleh **Process ID (PID)**. Fungsi **`fork()`** menciptakan duplikasi proses induk (proses anak). **`wait()`** digunakan oleh proses induk untuk menunda eksekusi hingga proses anak selesai (**sinkronisasi**).

IPC (Inter-Process Communication) adalah metode pertukaran data antar proses:

1. **Pipe**: Saluran satu arah, hanya berfungsi antara proses yang berkerabat (Parent-Child).
2. **FIFO (Named Pipe)**: Serupa dengan pipe, namun memiliki nama di sistem *file* sehingga dapat digunakan antar proses yang tidak berkerabat.
3. **Shared Memory (SHM)**: Mekanisme tercepat. Dua proses mengakses segmen memori fisik yang sama, menghilangkan *overhead* penyalinan data melalui kernel.

III. Percobaan dan Hasil

A. Percobaan Dasar Proses



```
yos@yos-QEMU-Virtual-Machine: ~ -- nano process_basic.c
GNU nano 8.4 process_basic.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

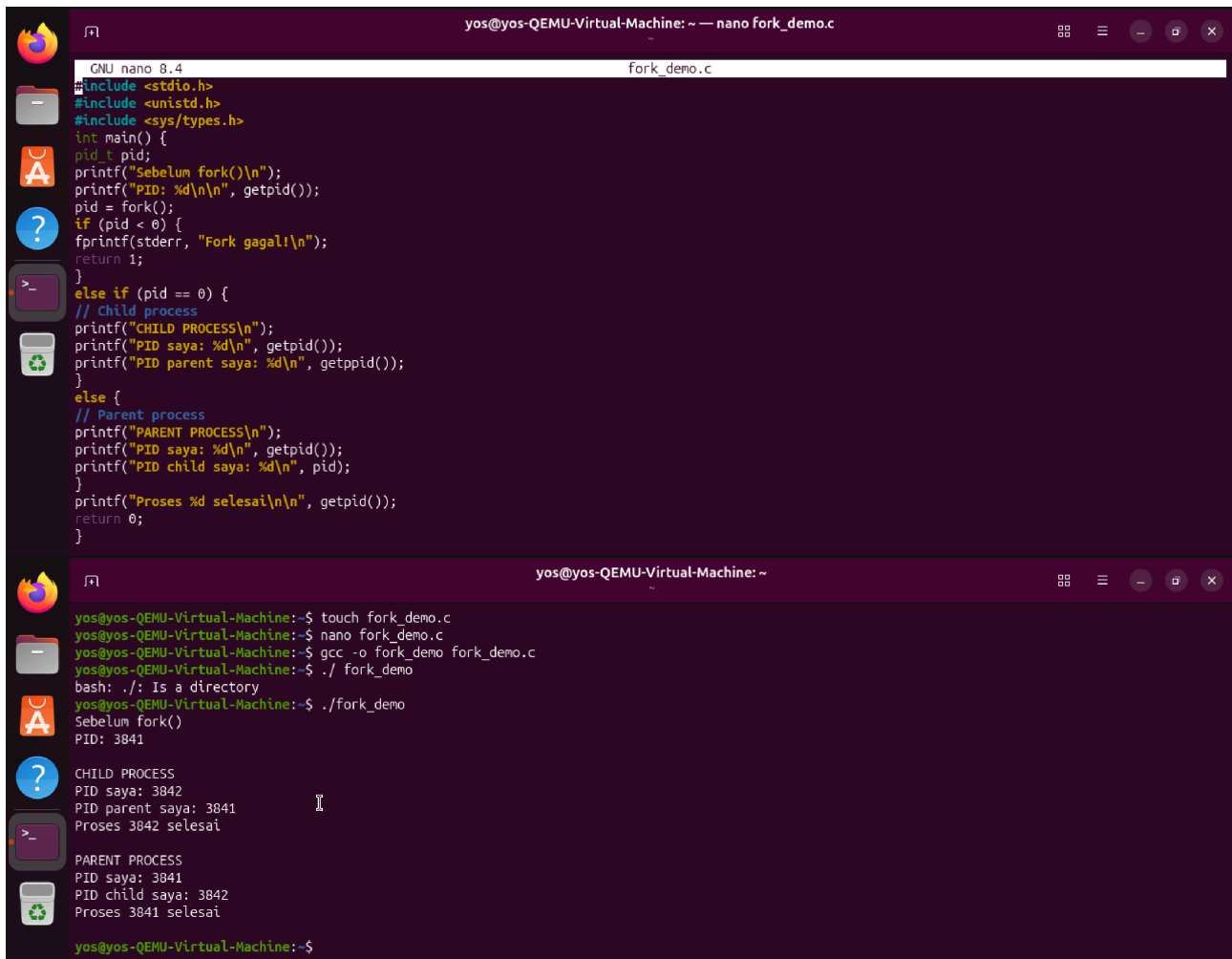
int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    return 0;
}

yos@yos-QEMU-Virtual-Machine: ~
yos@yos-QEMU-Virtual-Machine:~$ touch process_basic.c
yos@yos-QEMU-Virtual-Machine:~$ nano process_basic.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o process_basic process_basic.c
yos@yos-QEMU-Virtual-Machine:~$ ./process_basic
Program dimulai
PID proses ini: 3774
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Setiap eksekusi program menghasilkan sebuah proses dengan **PID** unik yang berhasil didapatkan menggunakan `getpid()`.

B. Percobaan Pembuatan Proses Anak (**fork**)



```
yos@yos-QEMU-Virtual-Machine: ~ -- nano fork_demo.c
GNU nano 8.4
fork_demo.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main() {
    pid_t pid;
    printf("Sebelum fork()\n");
    printf("PID: %d\n", getpid());
    pid = fork();
    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    }
    else {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }
    printf("Proses %d selesai\n", getpid());
    return 0;
}

yos@yos-QEMU-Virtual-Machine: ~
yos@yos-QEMU-Virtual-Machine:~$ touch fork_demo.c
yos@yos-QEMU-Virtual-Machine:~$ nano fork_demo.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o fork_demo fork_demo.c
yos@yos-QEMU-Virtual-Machine:~$ ./ fork_demo
bash: ./: Is a directory
yos@yos-QEMU-Virtual-Machine:~$ ./fork_demo
Sebelum fork()
PID: 3841

CHILD PROCESS
PID saya: 3842
PID parent saya: 3841
Proses 3842 selesai

PARENT PROCESS
PID saya: 3841
PID child saya: 3842
Proses 3841 selesai
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Fungsi `fork()` berhasil menduplikasi proses (PID 3841 menjadi PID 3842). Proses Induk (`pid > 0`) dan Anak (`pid == 0`) mengeksekusi kode secara konkuren.

C. Percobaan Sinkronisasi Proses (**wait**) dan Exit Code



```
yos@yos-QEMU-Virtual-Machine: ~ -- nano process_wait.c
GNU nano 8.4
process_wait.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    int status;
    pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }
    return 0;
}
```

```
yos@yos-QEMU-Virtual-Machine: ~  
yos@yos-QEMU-Virtual-Machine:~$ touch process_wait.c  
yos@yos-QEMU-Virtual-Machine:~$ nano process_wait.c  
yos@yos-QEMU-Virtual-Machine:~$ gcc -o process_wait process_wait.c  
yos@yos-QEMU-Virtual-Machine:~$ ./process_wait  
Parent: Menunggu child (PID: 3902) selesai...  
Child: PID saya 3902, akan tidur 3 detik  
Child: Selesai!  
Parent: Child selesai dengan exit code: 42  
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Proses Induk tertahan pada `wait()` hingga Proses Anak (PID 3902) selesai. Induk berhasil menangkap dan menampilkan **exit code 42** dari anak.

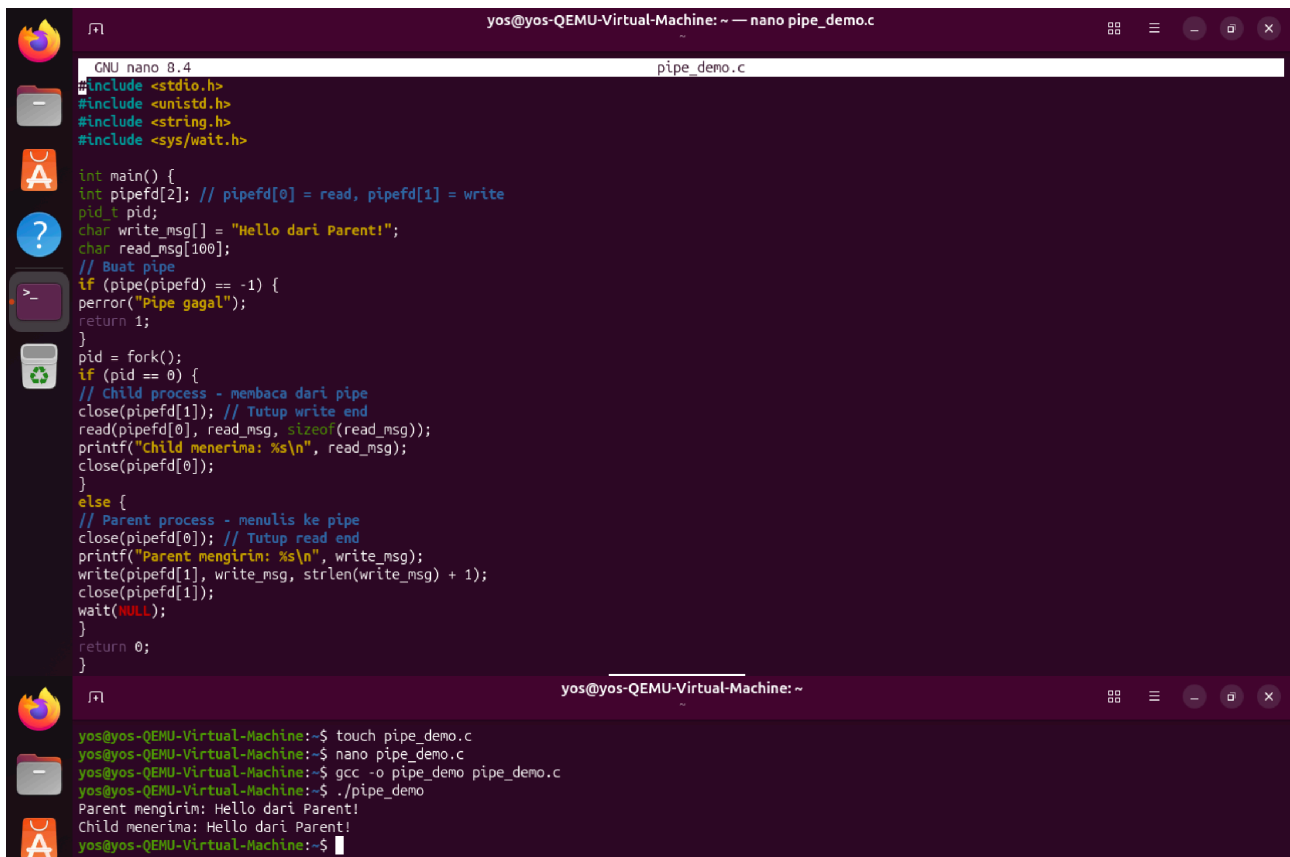
D. Percobaan Multiple **fork**

```
GNU nano 8.4 multiple_fork.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
int main() {  
    int i;  
    pid_t pid;  
    int num_children = 3;  
    printf("Parent PID: %d\n", getpid());  
    for (i = 0; i < num_children; i++) {  
        pid = fork();  
        if (pid == 0) {  
            // child process  
            printf("Child %d: PID = %d, Parent PID = %d\n",  
                i+1, getpid(), getppid());  
            sleep(i + 1); // Simulasi pekerjaan  
            printf("Child %d selesai\n", i+1);  
            return i;  
        }  
        // Parent menunggu semua child  
        for (i = 0; i < num_children; i++) {  
            int status;  
            pid_t child_pid = wait(&status);  
            printf("Parent: Child dengan PID %d selesai\n", child_pid);  
        }  
        printf("\nParent: Semua child selesai\n");  
        return 0;  
    }  
}
```

```
yos@yos-QEMU-Virtual-Machine:~$ touch multiple_fork.c  
yos@yos-QEMU-Virtual-Machine:~$ nano multiple_fork.c  
yos@yos-QEMU-Virtual-Machine:~$ gcc -o multiple_fork multiple_fork.c  
yos@yos-QEMU-Virtual-Machine:~$ ./multiple_fork  
Parent PID: 3956  
  
Child 1: PID = 3957, Parent PID = 3956  
Child 2: PID = 3958, Parent PID = 3956  
Child 3: PID = 3959, Parent PID = 3956  
Child 1 selesai  
Parent: Child dengan PID 3957 selesai  
Child 2 selesai  
Parent: Child dengan PID 3958 selesai  
Child 3 selesai  
Parent: Child dengan PID 3959 selesai  
  
Parent: Semua child selesai  
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Induk (PID 3956) berhasil mengelola dan menunggu penyelesaian ketiga proses anak secara sekuensial menggunakan `loop wait()`.

E. Percobaan Komunikasi Antar Proses (Pipe)



```
GNU nano 8.4 pipe_demo.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];
    // Buat pipe
    if (pipe(pipefd) == -1) {
        perror("Pipe gagal");
        return 1;
    }
    pid = fork();
    if (pid == 0) {
        // Child process - membaca dari pipe
        close(pipefd[1]); // Tutup write end
        read(pipefd[0], read_msg, sizeof(read_msg));
        printf("Child menerima: %s\n", read_msg);
        close(pipefd[0]);
    }
    else {
        // Parent process - menulis ke pipe
        close(pipefd[0]); // Tutup read end
        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);
        close(pipefd[1]);
        wait(NULL);
    }
    return 0;
}
```

```
yos@yos-QEMU-Virtual-Machine:~$ touch pipe_demo.c
yos@yos-QEMU-Virtual-Machine:~$ nano pipe_demo.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o pipe_demo pipe_demo.c
yos@yos-QEMU-Virtual-Machine:~$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Data berhasil ditransfer satu arah dari proses Induk (writer) ke proses Anak (reader) melalui *pipe*.

F. Percobaan FIFO (Named Pipe)



```
GNU nano 8.4 fifo_writer.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];
    // Buat FIFO jika belum ada
    mkfifo(FIFO_FILE, 0666);
    printf("Writer: Menunggu reader...\n");
    fd = open(FIFO_FILE, O_WRONLY);
    printf("Masukkan pesan: ");
    fgets(buffer, sizeof(buffer), stdin);
    write(fd, buffer, strlen(buffer) + 1);
    printf("Writer: Pesan terkirim\n");
    close(fd);
    return 0;
}
```

```
yos@yos-QEMU-Virtual-Machine: ~  
yos@yos-QEMU-Virtual-Machine:~$ gcc -o fifo_writer fifo_writer.c  
yos@yos-QEMU-Virtual-Machine:~$ ./fifo_writer  
Writer: Menunggu reader...  
saya yusa salasa  
pesan terkirim!  
^C  
yos@yos-QEMU-Virtual-Machine:~$
```

```
yos@yos-QEMU-Virtual-Machine: ~ — nano fifo_reader.c  
GNU nano 8.4  
fifo_reader.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <fcntl.h>  
#include <sys/stat.h>  
#include <unistd.h>  
#define FIFO_FILE "/tmp/my_fifo"  
int main() {  
    int fd;  
    char buffer[100];  
    printf("Reader: Membuka FIFO...\n");  
    fd = open(FIFO_FILE, O_RDONLY);  
    read(fd, buffer, sizeof(buffer));  
    printf("Reader: Menerima pesan: %s\n", buffer);  
    close(fd);  
    unlink(FIFO_FILE); // Hapus FIFO  
    return 0;  
}  
  
yos@yos-QEMU-Virtual-Machine: ~  
yos@yos-QEMU-Virtual-Machine:~$ touch fifo_reader.c  
yos@yos-QEMU-Virtual-Machine:~$ nano fifo_reader.c  
yos@yos-QEMU-Virtual-Machine:~$ gcc -o fifo_reader fifo_reader.c  
yos@yos-QEMU-Virtual-Machine:~$ ./fifo_reader  
Reader: Membuka FIFO...  
halo saya yusa salasa^C  
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Proses Writer **tertahan** saat membuka FIFO hingga Reader juga membukanya. Komunikasi berhasil terjalin antar dua proses yang tidak berkerabat melalui *file* FIFO di sistem.

G. Percobaan Shared Memory (SHM)

```
yos@yos-QEMU-Virtual-Machine: ~ — nano shm_writer.c  
GNU nano 8.4  
shm_writer.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <sys/ipc.h>  
#include <sys/shm.h>  
#define SHM_SIZE 1024  
int main() {  
    key_t key;  
    int shmid;  
    char *shared_memory;  
    char message[100];  
    // Generate unique key  
    key = ftok("shmfile", 65);  
    // Create shared memory  
    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);  
    // Attach to shared memory  
    shared_memory = (char*) shmat(shmid, NULL, 0);  
    printf("Writer: Masukkan pesan: ");  
    fgets(message, sizeof(message), stdin);  
    // Write to shared memory  
    strcpy(shared_memory, message);  
    printf("Writer: Data ditulis ke shared memory\n");  
    // Detach from shared memory  
    shmdt(shared_memory);  
    return 0;  
}  
  
yos@yos-QEMU-Virtual-Machine: ~  
yos@yos-QEMU-Virtual-Machine:~$ touch shm_writer.c  
yos@yos-QEMU-Virtual-Machine:~$ nano shm_writer.c  
yos@yos-QEMU-Virtual-Machine:~$ gcc -o shm_writer shm_writer.c  
yos@yos-QEMU-Virtual-Machine:~$ ./shm_writer  
Writer: Masukkan pesan: halo yusa salasa  
Writer: Data ditulis ke shared memory  
yos@yos-QEMU-Virtual-Machine:~$
```

```
yos@yos-QEMU-Virtual-Machine: ~ -- nano shm_reader.c
GNU nano 8.4 shm_reader.c *
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024
int main() {
    key_t key;
    int shmid;
    char *shared_memory;
    // Generate same key
    key = ftok("shmfile", 65);
    // Access shared memory
    shmid = shmget(key, SHM_SIZE, 0666);
    // Attach to shared memory
    shared_memory = (char*) shmat(shmid, NULL, 0);
    // Read from shared memory
    printf("Reader: Data dari shared memory: %s\n", shared_memory);
    // Detach and remove shared memory
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}

yos@yos-QEMU-Virtual-Machine: ~
yos@yos-QEMU-Virtual-Machine:~$ touch shm_reader.c
yos@yos-QEMU-Virtual-Machine:~$ nano shm_reader.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o shm_reader shm_reader.c
yos@yos-QEMU-Virtual-Machine:~$ ./shm_reader
Reader: Data dari shared memory: halo yusa salasa
yos@yos-QEMU-Virtual-Machine:~$
```

Analisis: Data berhasil dipertukarkan. Kedua proses melampirkan (*attach*) ke segmen memori fisik yang sama, memungkinkan akses data langsung tanpa perlu *kernel copy*.

H. Analisis Kinerja IPC (Benchmark)

```
yos@yos-QEMU-Virtual-Machine: ~ -- nano ipc_benchmark.c
GNU nano 8.4 ipc_benchmark.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h> // <-- penting untuk fungsi wait()

#define DATA_SIZE 1000000
#define SHM_SIZE (DATA_SIZE * sizeof(int))

// Fungsi untuk mendapatkan waktu dalam detik (floating)
double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

// Uji IPC menggunakan pipe()
void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int data[DATA_SIZE];
    double start, end;

    pipe(pipefd);
    start = get_time();
    pid = fork();

    if (pid == 0) {
        // Child process
        close(pipefd[1]);
    }
}
```

```
yos@yos-QEMU-Virtual-Machine: ~ — nano ipc_benchmark.c
GNU nano 8.4 ipc_benchmark.c
start = get_time();
pid = fork();

if (pid == 0) {
    // Child process
    close(pipefd[1]);
    read(pipefd[0], data, sizeof(data));
    close(pipefd[0]);
    exit(0);
} else {
    // Parent process
    close(pipefd[0]);
    for (int i = 0; i < DATA_SIZE; i++) {
        data[i] = i;
    }
    write(pipefd[1], data, sizeof(data));
    close(pipefd[1]);
    wait(NULL);
    end = get_time();
    printf("Pipe: %.6f detik\n", end - start);
}

// Uji IPC menggunakan shared memory
void test_shared_memory() {
    key_t key = ftok("shmfile", 65);
    int shmid;
    int *shared_data;
    pid_t pid;
    double start, end;

    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    start = get_time();

    if (pid == 0) {
        // Child process membaca shared memory
        shared_data = (int*) shmat(shmid, NULL, 0);
        long long sum = 0;
        for (int i = 0; i < DATA_SIZE; i++) {
            sum += shared_data[i];
        }
        shmdt(shared_data);
        exit(0);
    } else {
        // Parent process menulis ke shared memory
        shared_data = (int*) shmat(shmid, NULL, 0);
        for (int i = 0; i < DATA_SIZE; i++) {
            shared_data[i] = i;
        }
        wait(NULL);
        end = get_time();
        printf("Shared Memory: %.6f detik\n", end - start);
        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

int main() {
    printf("Benchmarking IPC Methods dengan %d integers\n\n", DATA_SIZE);
    test_pipe();
    test_shared_memory();
    return 0;
}

yos@yos-QEMU-Virtual-Machine:~$ touch ipc_benchmark.c
yos@yos-QEMU-Virtual-Machine:~$ nano ipc_benchmark.c
yos@yos-QEMU-Virtual-Machine:~$ gcc -o ipc_benchmark ipc_benchmark.c
yos@yos-QEMU-Virtual-Machine:~$ ./ipc_benchmark
Benchmarking IPC Methods dengan 1000000 integers
```

Detail Implementasi Kinerja:

1. **Pipe:** Melibatkan dua kali penyalinan data penuh (*user* $\rightarrow$ *kernel* $\rightarrow$ *user*), yang menambah *overhead*.
2. **Shared Memory:** Hanya melibatkan satu kali penulisan ke SHM oleh Induk. Anak membaca data **langsung** dari segmen memori bersama.

Analisis: Berdasarkan prinsip IPC, Shared Memory jauh lebih cepat dari Pipe karena menghindari *system call* dan *data copying* oleh kernel.

IV. Kesimpulan

Berdasarkan serangkaian percobaan yang telah dilakukan, dapat disimpulkan beberapa poin penting mengenai Manajemen Proses dan Komunikasi Antar Proses (IPC) pada sistem operasi Linux:

1. Manajemen Proses Dasar

- **Proses dan PID:** Setiap program yang dieksekusi merupakan sebuah proses yang diidentifikasi secara unik oleh **Process ID (PID)**. Fungsi `getpid()` berhasil digunakan untuk mengambil PID proses yang sedang berjalan.
- **Pembentukan Proses (`fork`):** Fungsi `fork()` berhasil menduplikasi proses induk menjadi proses anak, memungkinkan eksekusi kode secara **konkuren**.
- **Sinkronisasi (`wait`):** Fungsi `wait()` sangat krusial untuk sinkronisasi, memastikan proses induk dapat **menunggu** penyelesaian proses anak dan berhasil mengambil **exit code** yang dikembalikan oleh proses anak.

2. Mekanisme Komunikasi Antar Proses (IPC)

Percobaan menunjukkan beberapa mekanisme IPC dengan karakteristik berbeda:

- **Pipe (Unnamed Pipe):** Berhasil digunakan untuk transfer data **satu arah** (unidirectional) secara sederhana antara proses yang berkerabat (induk dan anak).
- **FIFO (Named Pipe):** Berhasil memfasilitasi komunikasi **satu arah** antara proses-proses yang **tidak berkerabat** (independen) dengan menggunakan entri yang dinamakan di sistem *file*.
- **Shared Memory (SHM):** Merupakan mekanisme IPC yang paling efisien karena memungkinkan dua proses mengakses segmen memori fisik yang sama.

3. Perbandingan Kinerja IPC

Analisis kode *benchmark* menunjukkan perbedaan fundamental dalam implementasi IPC yang memengaruhi kinerja:

- Mekanisme berbasis *kernel* seperti **Pipe** melibatkan *overhead* yang signifikan karena membutuhkan **penyalinan data** melalui *kernel space*.
- Mekanisme **Shared Memory** memberikan kinerja superior, terutama untuk transfer data besar, karena data diakses secara **langsung** dari memori bersama, **meminimalkan *system call*** dan **eliminasi *data copying*** oleh kernel.